

activityProcessing.ts — Telemetry Processing Service

`processActivity(input)` is the single exported function. It runs five deterministic steps:

1. **Transport** — `distanceKm × TRANSPORT_FACTORS_KG_KM[mode]`. The factor map uses DEFRA 2024/2025 TTW+WTT values and defaults to `0.21` (petrol car) for any unknown mode, which is the safest conservative fallback.
2. **Diet** — A direct lookup into the Scarborough et al. (2014) daily constants. Your schema uses `"mixed"` where the blueprint calls it `"medium_meat"` — those are mapped in the constant object so the rest of the codebase doesn't need to care.
3. **Home energy** — `E_elec = C_kWh × EF_grid`. When no live grid intensity is passed (i.e., when the Emissions.dev API call hasn't resolved yet), it falls back to India's coal-heavy average of `0.82 kg CO2e/kWh` — the worst-case figure from the blueprint, which is the correct conservative default for Indian users.
4. **Delta vs baseline** — `carbonDeltaKg = total - baseline`. Negative means the user beat their baseline. `carbonSavedKg` is clamped to `0` on days they exceed it so the DB never stores a negative "saved" figure.
5. **Gamification** — Points are modular. Base log → diet bonus → zero-emission transport bonus → below-baseline bonus → streak multiplier (capped at 30 days to prevent compounding runaway).

All floats are rounded to 4 decimal places before return to prevent floating-point drift accumulating across DB aggregation queries.

insightEngine.ts — Behavioral Nudge Engine

`generateDailyNudge(recentLogs)` evaluates the last 3 logs through a **priority cascade** — first match wins, preventing nudge spam:

Priorit y	Trigger	Framework	Savings
1	≥2 of 3 days logged <code>high_meat</code>	Gamification + Commitment Consistency	4.30 kg/day

2	≥2 of 3 trips by car, all <15 km	Salience / Choice Architecture	Dynamic per trip
3	All logs used car, no green mode used	Default Bias	0.80 kg/10 km

Each nudge returns a `triggerContext` object for A/B testing and logging — you can store this alongside the nudge in a `NudgeLog` collection if needed.

aiCoach.ts — LLM System Prompt

The system prompt has two hard constraints baked in that prevent the most common LLM failure modes:

- **Anti-hallucination rules** are explicit: the model may only reference numbers that exist in the JSON. If a field is null or missing, it must not mention it. This is specified in imperative terms, not suggestions.
- **Sentence 1 must cite a specific figure** from the data — this forces grounding on the first token and makes constraint violations immediately visible in QA.

`buildWeeklyCoachPrompt()` does one piece of server-side pre-computation: it identifies the `worstEmissionCategory` and injects it into the payload. This means the model always has an explicit anchor for its third sentence (the actionable directive), removing ambiguity about what to focus on.